

Project # 1 : Credit Card Fraud Detection

Baudart Alexandre

Last update : June 9, 2026

Contents

1	Dataset and Exploratory Analysis	3
1.1	Dataset Overview	3
1.2	Features Description	3
1.3	Missing Values	4
1.4	Correlation Analysis	4
2	Preprocessing and Modeling	6
2.1	Data Splitting and Preprocessing	6
2.2	Models	6
2.3	Hyperparameter Optimization	7
2.4	Evaluation Protocol	9
3	Results and Discussion	10
3.1	Classical Machine Learning Models	10
3.2	Confusion Matrix Analysis	11
3.3	Visual Performance Analysis	12
3.4	Multi-Layer Perceptron Experiments	12
3.4.1	Validation Strategy and Loss Functions	12
3.4.2	Imbalance Handling Experiments	13
3.4.3	Architectural Ablation Studies	13
3.4.4	Final MLP Performance	14
4	Conclusion	15

1 Dataset and Exploratory Analysis

1.1 Dataset Overview

The dataset **Credit Card Faud Detection** [3] has been used for this project. It was created in the context of the research works of *Dal Pozzolo et al.* [2, 1] over the extreme class unbalancing.

This dataset is about credit card fraud detection. It includes anonymized transactions made by European credit card holders.

The dataset contains 284,807 transactions going along with a binary label :

- class 0 : legitimate transaction
- class 1 : fraudulent transaction

Nevertheless, this dataset is **extremely unbalanced**. Indeed, the fraudulent transactions represent only 0.18% of the whole dataset (492 examples) against 284,315 examples of legitimate transactions.

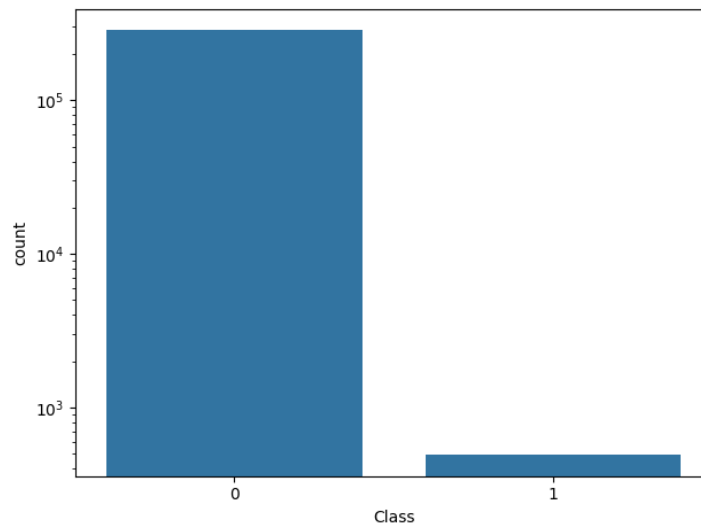


Figure 1: Countplot of the target variable *Class* (logarithm scale)

This tremendous unbalancing was one of the main challenges of the project and justify the using of adapted metrics like PR-AUC.

1.2 Features Description

The dataset includes 30 explanatory variables (features) and 1 binary target variable (label).

Among those features :

- *Time* represents the number of seconds elapsed between this transaction and the first one in the dataset.
- *Amount* is the amount of the transaction.

Unfortunately, due to confidentiality issues, the 28 others original features are not provided. Instead, these 28 features (*V1* to *V28*) are components transformed by PCA. Consequently, we don't have a lot of background information about the data, making the interpretation of the results quite tough.

1.3 Missing Values

There is not any missing value in this dataset, that simplified the preprocessing.

1.4 Correlation Analysis

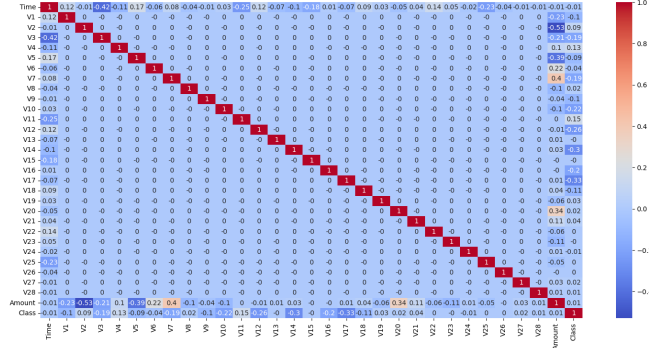


Figure 2: Correlation analysis including the target variable (*Class*)

A correlation analysis has been produced in order to identify the most related variables to the target variable.

Results show that any individual variable is not significantly correlated with the target. This suggests that fraud detection could be based on complex interactions between several variables rather than one dominant feature.

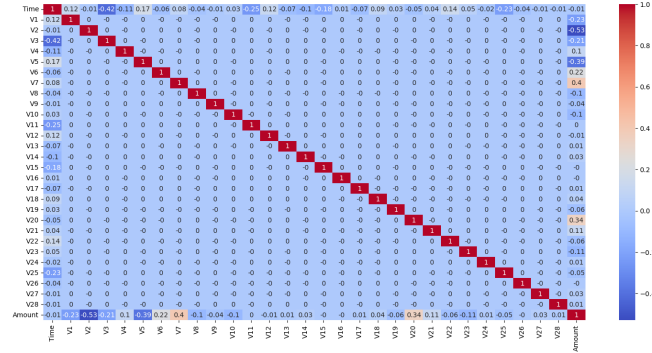


Figure 3: Correlation analysis excluding the target variable

An other analysis, excluding the target variable, shows that there are some moderate correlations between the features :

- $V2$ presents a negative correlation about -0.53 with $Amount$.
- $V7$ shows a positive moderate correlation around 0.40 with $Amount$.

However, the reading of those relations remains limited because the PCA components does not have an explicit business meaning.

2 Preprocessing and Modeling

2.1 Data Splitting and Preprocessing

The dataset was split into training and test sets using an 80/20 stratified split in order to preserve the original class distribution across all subsets. Given the extreme class imbalance of the dataset, stratification was essential to ensure representative distributions of both classes in each split.

All classical machine learning models were evaluated on the train dataset using stratified k-fold cross-validation in order to ensure robust performance estimation and stable variance across folds.

For the MLP model, a single stratified train/validation split was used (20% of the training set), generated via `StratifiedShuffleSplit`, and preserved throughout training. The validation set was used for monitoring performance during training, early stopping, and learning rate scheduling. Moreover, using a k-fold cross-validation would have been impractical due to the high computational cost of the neural network training and the cross-validation as well.

Furthermore, feature preprocessing was applied depending on the model type :

- Standardization was applied for Logistic Regression, SVM and the MLP model.
- Tree-based models (Random Forest and XGBoost) were trained on raw features without normalization, as they are invariant to monotonic transformations.

2.2 Models

The following models were evaluated in this study :

- Logistic Regression
- Linear SVM (trained via SGD)
- Random Forest
- XGBoost
- Multi-Layer Perceptron (MLP)

The selection covers a wide range of model families, from linear models to ensemble methods and deep learning architectures.

2.3 Hyperparameter Optimization

Hyperparameter tuning was performed using :

- Optuna for classical machine learning models.
- Ray Tune for the MLP model.

The best configurations obtained are summarized below.

LOGISTIC REGRESSION

Parameter	Value
C	0.01
max_iter	716
intercept_scaling	0.69
fit_intercept	True
warm_start	True
tol	1e-6

Table 1: Main parameters for logistic regression

SVM (SGD-BASED)

Parameter	Value
alpha	0.0003
penalty	L1
loss	squared_hinge
learning_rate	optimal
tol	1e-3

Table 2: Main parameters for SVM

RANDOM FOREST

Parameter	Value
n_estimators	355
max_depth	7
min_samples_split	2
min_samples_leaf	8
oob_score	True

Table 3: Main parameters for random forest

XGBoost

Parameter	Value
n_estimators	2000
max_depth	6
min_child_weight	8
subsample	0.99
colsample_bytree	0.88
reg_alpha	3.60
reg_lambda	0.04

Table 4: Main parameters for XGBoost

Training configuration :

- early stopping based on PC-AUC evolution (rounds = 50)

MULTI-LAYER PERCEPTRON (MLP)

Parameter	Value
n_layers	3
hidden_size	128
dropout	0.08
initial learning rate	1e-3

Table 5: Main parameters for MLP

Training configuration :

- batch size = 64,
- loss function = BCEWithLogitsLoss,
- optimizer = AdamW,
- gradient clipping enabled,
- learning rate scheduling using, ReduceLROnPlateau
- early stopping based on validation loss.

2.4 Evaluation Protocol

Two evaluation phases were used in this study.

CROSS-VALIDATION PHASE

For classical machine learning models, performance was assessed using stratified k-fold cross-validation. The reported PR-AUC corresponds to the **mean and standard deviation across folds**, providing a robust estimate of model performance and stability.

Note : Cross-validation was not used with MLP but instead a validation phase was set up to monitor performance, apply early stopping, and adjust learning rate scheduling.

TEST PHASE

Final model performance was evaluated on a held-out test set using the following metrics :

- Precision
- Recall
- F1-score
- ROC-AUC
- PR-AUC

3 Results and Discussion

3.1 Classical Machine Learning Models

The performances obtained by the classical machine learning models are summarized below.

Model	Train CV PR- AUC (Mean)	Train CV PR- AUC (Std)	Precision	Recall	F1- Score	ROC- AUC	PR- AUC
Logistic Regres- sion	0.76	0.04	0.82	0.61	0.70	0.80	0.50
SGD- SVM	0.64	0.07	0.72	0.65	0.68	0.82	0.47
Random Forest	0.82	0.03	0.93	0.78	0.85	0.89	0.73
XGBoost	0.84	0.016	0.94	0.81	0.87	0.90	0.77

Table 6: Classical machine learning models results

The results show a clear performance gap between linear models and non-linear ensemble methods.

Logistic regression and SGD-SVM achieved acceptable precision scores but struggled to correctly identify fraudulent transactions, resulting in lower recall and PR-AUC values. These results suggest that fraud detection in this dataset depends on complex non-linear interactions that cannot be fully captured by linear decision boundaries.

Tree-based ensemble methods significantly outperformed linear approaches. Both Random Forest and XGBoost achieved strong precision-recall trade-offs while maintaining high PR-AUC scores and low variance across cross-validation folds.

Among all classical machine learning models, XGBoost obtained the best overall performance, achieving :

- the highest PR-AUC,
- the highest F1-Score,
- and the most stable cross-validation behavior.

These observations confirm the effectiveness of gradient boosting methods on highly imbalanced tabular datasets.

3.2 Confusion Matrix Analysis

The confusion matrices obtained on the test set provide additional insight into the behavior of each model under severe class imbalance.

Model	TN	FP	FN	TP
Logistic Re- gression	56851	13	38	60
SGD-SVM	56839	25	34	64
Random For- est	56859	5	21	77
XGBoost	56859	13	38	90
MLP	56848	16	19	79

Table 7: Confusion matrices results

Linear models produced substantially more false negatives, meaning that a larger number of fraudulent transactions remained undetected.

Although Logistic Regression achieved relatively high precision, its lower recall indicates difficulties in detecting minority-class samples. The SGD-based SVM slightly improved recall but at the cost of a larger number of false positives.

Random Forest and XGBoost considerably improved fraud detection capability while maintaining very low false positive rates.

Among all evaluated models, XGBoost achieved the best compromise :

- only 18 false negatives,
- only 5 false positives,
- and the highest PR-AUC score.

The MLP achieved performance very close to XGBoost (cf. part 3.4.4), demonstrating strong capability in capturing complex patterns despite the severe imbalance of the dataset.

From an application perspective, minimizing false negatives is particularly important in fraud detection systems since undetected fraudulent transactions may lead to direct financial losses. Under this perspective, XGBoost and the MLP provided the most effective trade-offs.

3.3 Visual Performance Analysis

Since XGBoost achieved the best overall performance, ROC and Precision-Recall curves are presented below in order to further illustrate its classification behavior under severe class imbalance.

Due to the highly imbalanced nature of the dataset, Precision-Recall curves provide more informative insights than ROC curves regarding fraud detection capability.

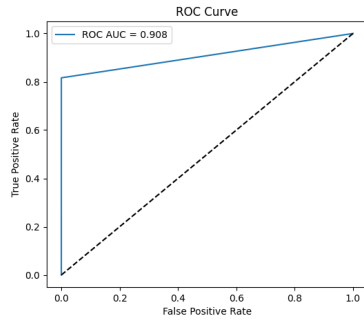


Figure 4: XGBoost ROC-AUC Curve

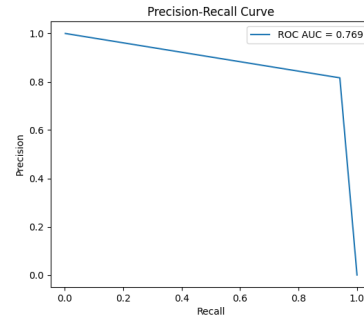


Figure 5: XGBoost PR-AUC Curve

3.4 Multi-Layer Perceptron Experiments

Several experiments were conducted to evaluate the impact of :

- validation strategies,
- loss functions,
- imbalance handling techniques,
- architectural components,
- and regularization methods.

3.4.1 Validation Strategy and Loss Functions

The best MLP configuration was obtained using a stratified validation split generated with `StratifiedShuffleSplit` and `BCEWithLogitsLoss` as the training criterion.

This configuration achieved :

- Train PR-AUC : 0.8085
- Validation PR-AUC : 0.7689

Experiments using focal loss generally produced less stable optimization behavior and lower validation PR-AUC scores.

These results suggest that, despite the strong imbalance of the dataset, standard binary cross-entropy remained more effective and more stable than focal loss in this setting.

3.4.2 Imbalance Handling Experiments

Additional experiments were conducted using :

- pos.weight,
- WeightedRandomSampler,
- and combinations of both methods.

Contrary to expectations, these techniques did not improve validation performance and frequently introduced unstable training dynamics.

In particular, the use of `WeightedRandomSampler` led to near-perfect training PR-AUC values while substantially degrading validation performance, indicating severe overfitting and memorization of minority-class samples.

These observations highlight that imbalance handling techniques must be applied carefully, as aggressive reweighting strategies may negatively impact generalization.

3.4.3 Architectural Ablation Studies

Several architectural components were evaluated through ablation experiments.

SKIP CONNECTIONS

Removing skip connections produced only minor performance differences, suggesting that residual connections were not critical for this relatively shallow architecture.

BATCH NORMALIZATION

Removing the initial batch normalization layer caused validation PR-AUC to collapse below 0.05.

This experiment demonstrated that batch normalization was essential for stable optimization and convergence.

LAYER NORMALIZATION

Removing layer normalization slightly reduced validation performance while preserving relatively stable training behavior.

This suggests that layer normalization contributed positively to generalization but was less critical than batch normalization.

DROPOUT EXPERIMENTS

Different dropout values were evaluated : 0.0, 0.08, 0.18 and 0.28.

Moderate dropout slightly improved generalization, while excessive dropout degraded validation performance.

Overall, dropout had a relatively limited impact compared to normalization layers and validation strategy.

3.4.4 Final MLP Performance

Metric	Value
Train PR-AUC	0.8085
Validation PR-AUC	0.7689
Precision	0.83
Recall	0.83
F1-Score	0.80
ROC-AUC	0.97
Test PR-AUC	0.76

Table 8: MLP results

The final MLP achieved performance comparable to XGBoost while requiring substantially more experimentation and training stabilization techniques.

Its strong PR-AUC and recall values demonstrate that neural networks can effectively model complex fraud detection patterns when appropriate validation strategies and regularization mechanisms are used.

4 Conclusion

The experiments conducted throughout this study highlight several important observations.

First, linear models were insufficient for capturing the complex relationships underlying fraudulent transactions. Their lower recall and PR-AUC scores suggest that fraud detection in this dataset requires non-linear modeling capabilities.

Second, tree-based ensemble methods proved extremely effective for this task. In particular, XGBoost consistently achieved the best balance between precision, recall, and overall robustness.

Third, the MLP experiments demonstrated that training stability plays a central role in deep learning approaches for tabular imbalanced datasets. Validation strategy, normalization layers, and regularization mechanisms had a significant impact on performance.

Overall, XGBoost achieved the strongest and most stable results while remaining computationally efficient. Nevertheless, The MLP achieved highly competitive performance and demonstrated strong representation learning capabilities on this challenging fraud detection task.

References

- [1] Andrea Dal Pozzolo, Giacomo Boracchi, Olivier Caelen, Cesare Alippi, and Gianluca Bontempi. Credit card fraud detection: a realistic modeling and a novel learning strategy. IEEE Transactions on Neural Networks and Learning Systems, 29(8):3784–3797, 2018.
- [2] Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson, and Gianluca Bontempi. Calibrating probability with undersampling for unbalanced classification. In IEEE Symposium on Computational Intelligence and Data Mining (CIDM). IEEE, 2015.
- [3] Worldline and Machine Learning Group de l’ULB. Credit Card Fraud Detection Dataset. Dépôt Kaggle, 2016. Disponible sur : <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>.